Lecture 22 → Touch the Network: Scapy, Mininet, and Sockets

Arc of the day

1. Scapy → craft a single packet, byte by byte
2. Mininet → own a whole topology
3. Sockets → the interface real applications use

What is scapy? → A Python library for building, sending, receiving, and dissecting packets at any layer
Why use it → Fastest way to test a network hypothesis. “What does this router do with a packet I built by hand?”

Scapy → Setup

```
sudo pip install scapy
sudo python3    # raw sockets require root
```

Scapy → Building a packet

```
from scapy.all import IP, ICMP

pkt = IP(dst="8.8.8.8") / ICMP()
pkt.show()          # pretty-print every field
bytes(pkt)          # the actual wire bytes
```

Scapy → Ping from scratch

```
from scapy.all import IP, ICMP, sr1

req = IP(dst="8.8.8.8") / ICMP()
reply = sr1(req, timeout=2, verbose=0)
reply.show()
```

Scapy → TCP SYN Probe

```
from scapy.all import IP, TCP, sr1, RandShort

r = sr1(IP(dst="scanme.nmap.org")
        / TCP(dport=80, flags="S", sport=RandShort()),
        timeout=2, verbose=0)
print(r.sprintf("%TCP.flags%"))   # 'SA' = SYN-ACK = open
```

Scapy → traceroute

```
from scapy.all import IP, ICMP, sr1

for ttl in range(1, 16):
    r = sr1(IP(dst="8.8.8.8", ttl=ttl) / ICMP(),
            timeout=2, verbose=0)
    if r is None:
        print(f"{ttl:2d}␣␣*")
    elif r.type == 0:                    # Echo Reply = reached
        print(f"{ttl:2d}␣␣{r.src}␣␣(reached)"); break
    else:                                # Time Exceeded
        print(f"{ttl:2d}␣␣{r.src}")
```

Scapy → Sniff mode

```
from scapy.all import sniff

pkts = sniff(filter="icmp", count=5, iface="any")
pkts.summary()
```

What is mininet ?

Mininet creates virtual hosts using Linux network namespaces and connects them with virtual Ethernet pairs (veth)

- Each “host” is a real Linux process with its own network stack
- It is not a simulation → it is a real network, in miniature, on one kernel
- Ten lines of Python → a topology with controllable bandwidth, delay, and loss

This is how researchers (and engineers at Google, Cisco, etc.) test protocol changes before shipping them to physical routers.

Mininet → hello world

```
sudo mn --topo single,3 --link tc

mininet> h1 ping -c 3 h2
mininet> pingall
mininet> exit
```

Mininet → bottle nech topology

```
from mininet.net import Mininet
from mininet.link import TCLink
from mininet.node import OVSController

net = Mininet(controller=OVSController, link=TCLink)
net.addController('c0')
h1 = net.addHost('h1'); h2 = net.addHost('h2')
s1 = net.addSwitch('s1')

net.addLink(h1, s1, bw=100, delay='1ms')
net.addLink(s1, h2, bw=10, delay='20ms', loss=1)   # bottleneck

net.start()
h2.cmd('iperf3␣-s␣-D')
print(h1.cmd('iperf3␣-c␣%s␣-t␣20' % h2.IP()))
net.stop()
```

<u>Mininet → what to watch for</u>

- Bandwidth: settles near 10 Mb/s — bottleneck governs
- Loss: 1% triggers fast retransmit and multiplicative decrease (L20); throughput noticeably below the loss-free ceiling
- Delay: 20 ms one-way ⇒ 40 ms RTT
- BDP: 10 Mb/s × 40 ms ≈ 50 KB ≈ 35 MSS

<u>Socket API</u>

Every networked program you have ever used sits on top of this API: browsers, SSH, dig, curl, Zoom, game

TCP server flow: socket() → bind() → listen() → accept() → recv()/send() → close()

TCP client flow: socket() → connect() → send()/recv() → close()

UDP server flow: socket() → bind() → recvfrom()/sendto() → close()

UDP client flow: socket() → sendto()/recvfrom() → close()

<u>TCP vs UDP API</u>

	TCP	UDP
Socket type	SOCK_STREAM	SOCK_DGRAM
Connection setup	listen/accept/connect	none
Message boundaries	byte stream (framed by app)	preserved per datagram
Retransmission	kernel	application's job
Ordering	guaranteed	none
Typical users	HTTP, SSH, Git	DNS, QUIC, video, games

TCP Echo → server

```
import socket
HOST, PORT = '127.0.0.1', 5001

srv = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
srv.bind((HOST, PORT))
srv.listen(1)

while True:
    conn, addr = srv.accept()
    with conn:
        while True:
            data = conn.recv(1024)
            if not data: break
            conn.sendall(data)
```

TCP Echo → client

```
import socket

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.connect(('127.0.0.1', 5001))
    s.sendall(b'hello␣from␣ec441\n')
    print(s.recv(1024))
```

UDP Echo → server and client

Server

```
import socket
srv = socket.socket(socket.AF_INET,
                    socket.SOCK_DGRAM)
srv.bind(('127.0.0.1', 5002))
while True:
    data, addr = srv.recvfrom(1024)
    srv.sendto(data, addr)
```

Client

```
import socket
s = socket.socket(socket.AF_INET,
                  socket.SOCK_DGRAM)
s.sendto(b'hello␣udp\n',
         ('127.0.0.1', 5002))
print(s.recvfrom(1024))
```

Setsockopt → options worth knowing

Option	Effect	Set when
SO_REUSEADDR	Immediate rebind after close	always on servers
TCP_NODELAY	Disable Nagle's algorithm	interactive / RPC
SO_KEEPALIVE	Periodic probes on idle TCP	long-lived connections
SO_RCVBUF / SNDBUF	Socket buffer size	high-BDP paths

Close the loop

```
# shell 1 -- capture
sudo tcpdump -i lo -w /tmp/sockets.pcap \
    'port 5001 or port 5002'

# shell 2 -- servers
python3 demo_tcp_echo_server_l22.py &
python3 demo_udp_echo_server_l22.py &

# shell 3 -- clients
python3 demo_tcp_echo_client_l22.py
python3 demo_udp_echo_client_l22.py

# shell 1 -- stop
sudo pkill tcpdump
```

Every layer , one run

The sockets.pcap you just captured contains:

- Ethernet frame (L08) carrying
- IP datagram (L14–L17) carrying
- TCP segment with cwnd, seq/ack, RTT (L18–L20) carrying
- application bytes from your Python code